Examen final — Fase 1: roles que regulan el consumo de las APIs

Rama: `final-fase-1-roles` (reaplicada como `merge/final-fase-1-roles` sobre `origin/main`; ver la sección 8) · **Fecha:** 12/11/2026 · **Estado:** implementada, probada, **integrada en `main` (PR #18, 12/11/2026)** y desplegada
Consigna (PDF, «Requisitos para el examen final», punto 2): *«Definir e implementar roles de usuario que regulen el consumo de las APIs»*.

1. Qué se hizo

Cada endpoint exige ahora un rol. Antes bastaba con tener cualquier token válido: el filtro JWT ya reconstruía los roles del token (`JWTAuthorizationFilter`, líneas 73 a 77) y `@EnableMethodSecurity` estaba activo, pero ningún método los usaba.

Roles (decisión aprobada por el usuario el 12/11/2026): tres personas (`ROLE_ADMIN`, `ROLE_SUPERVISOR`, `ROLE_OPERADOR`) y tres sistemas externos (`ROLE_SAP`, `ROLE_TMS`, `ROLE_LOADER`). `ROLE_USER` sigue existiendo en la base pero no tiene permiso sobre nada.

Endpoint	ADMIN	SUPERVISOR	OPERADOR	SAP	TMS	LOADER
`POST /integration/sap/orders` (Punto 1)	✓			✓		
`POST /integration/tms/orders/{n}/tare` y `/final-weight` (Puntos 2 y 5)	✓				✓	
`POST /integration/loader/activation` y `/orders/{n}/details` (Punto 3)	✓					✓
`POST /integration/loader/orders/{n}/close` (Punto 4)	✓	✓	✓			✓
`GET /orders`, `/{n}`, `/{n}/details`, `/{n}/status-log`, `/{n}/reconciliation`	✓	✓	✓			
`GET /demo/encodepass`	✓					
`POST /login`	público					

`ADMIN` accede a todo, así la colección Postman anterior y el simulador del frontend siguen funcionando. Sin token sigue siendo **401**; con token pero sin el rol, **403**.

2. Inventario de archivos

Archivo	Acción	Motivo
`security/RestAccessDeniedHandler.java`	**Nuevo**	Responde el 403 con el JSON estándar `{ message, code, devInfo }`
`security/SecurityConfiguration.java`	Modificado (+5)	Registra el handler: `http.exceptionHandling(e -> e.accessDeniedHandler(...))`
`controllers/OrderRestController.java`	Modificado	`@PreAuthorize` en los 5 GET
`integration/sap/controllers/OrderSapRestController.java`	Modificado	`@PreAuthorize("hasAnyRole('ROLE_ADMIN', 'ROLE_SAP')")`
`integration/tms/controllers/OrderTmsRestController.java`	Modificado	Ídem para `ADMIN` y `TMS`

Archivo	Acción	Motivo
`integration/loader/controllers/OrderLoaderRestController.java`	Modificado	`ADMIN` y `LOADER` (activación, lectura); más `SUPERVISOR` y `OPERADOR` (cierre)
`auth/controller/AuthRestController.java`	Modificado	`@PreAuthorize("hasRole('ROLE_ADMIN')")` en `/demo/encodepass`
`src/test/.../security/RolesAccessTest.java`	**Nuevo**	98 pruebas de la matriz
`docs/sql/seed-dev-users.sql`	Modificado	Los 6 roles y un usuario por rol (idempotente)
`docs/postman/IW3-Gas-Terminal.postman_collection.json`	Modificado (solo agrega)	Carpeta `08 - Roles (examen final)`
`docs/final-fase-1-roles.md`	**Nuevo**	Este informe

Los 12 métodos anotados también declaran `@ApiResponse(responseCode = "403", ...)` en Swagger. No se tocó ninguna clase de negocio ni de modelo.

3. Recorrido de un pedido (el caso `operador` intenta crear una orden)

1. `POST /api/v1/integration/sap/orders` con `Authorization: Bearer <jwt de operador>`.
2. `JWTAuthorizationFilter` verifica la firma y el vencimiento, arma un `User` con el nombre y las autoridades `[ROLE_OPERADOR]` y lo deja en el `SecurityContextHolder`.
3. La regla `anyRequest().authenticated()` lo deja pasar: está autenticado.
4. Spring MVC enlaza el cuerpo JSON a `Order` y llama al método del controller **a través de un proxy**.
5. El proxy evalúa `@PreAuthorize("hasAnyRole('ROLE_ADMIN', 'ROLE_SAP')")` contra las autoridades: no coincide → lanza `AccessDeniedException` **antes de entrar al método**. Por eso el `try/catch` del controller no la ve.
6. `ExceptionTranslationFilter` la atrapa y llama a `RestAccessDeniedHandler`, que responde **403** con `{"message":"No tiene permisos para realizar esta operación","code":403,"devInfo":null}` (en producción; en desarrollo `devInfo` trae la traza).

4. Decisiones de diseño

#	Decisión	Por qué
R-1	`@PreAuthorize` en cada método del controller, no reglas por URL en `SecurityConfiguration`	Es lo que enseña la cátedra (clase 9, `MailRestController`) y deja el permiso visible junto al endpoint y en Swagger
R-2	`hasAnyRole('ROLE_…')` con el prefijo escrito	Igual que la cátedra; Spring no duplica el prefijo
R-3	Handler propio para el 403, con JSON armado a mano	Sin él, el 403 sale vacío y el frontend no tiene `message` que mostrar. Se arma a mano porque el proyecto no usa `ObjectMapper` y Spring Boot 4 trae dos Jackson
R-4	El mensaje del 403 no nombra roles	No le dice a un atacante qué rol buscar
R-5	`ADMIN` en todas las filas	Compatibilidad con Postman y con el simulador del frontend
R-6	Los tests recorren la cadena real con JWT firmados con `AuthConstants.SECRET`	Prueban el filtro, la anotación y el handler juntos, sin base de datos (`@WebMvcTest` + negocio simulado)

5. Pruebas ejecutadas (12/11/2026)

Prueba	Resultado
`./mvnw -B test`	**138 tests, 0 fallas** (40 anteriores + 98 de `RolesAccessTest`)
`RolesAccessTest`, primero **sin** las anotaciones	55 fallas y 1 error: los casos «denegado» fallaban, como debía

Prueba	Resultado
`RolesAccessTest`, con la implementación	**98 de 98**: 12 endpoints × 7 roles (incluido `ROLE_USER`), 12 pedidos sin token (401), un rol desconocido (403) y el formato del 403
Matriz contra el backend real (6 logins reales, 12 endpoints, orden inexistente)	**72 de 72** correctos; sin token 401; el 403 llega como `application/json`
Seed aplicado dos veces	6 usuarios y 6 asignaciones: idempotente
Postman, colección completa con newman	**186 pedidos, 428 aserciones, 0 fallas** (línea base 154 y 364; la carpeta 08 agrega 32 y 64)

Hallazgo de las pruebas: con el cuerpo `{}` el endpoint SAP responde **400**, no 403, porque `Order.preset` es `double` primitivo y Jackson 3 rechaza el campo ausente **antes** de que el proxy compruebe el rol. No es una falla de seguridad (el 400 no revela datos) pero explica por qué los tests mandan `{"preset":1}`.

6. Cómo probarlo a mano

```
docker compose up -d && ./mvnw spring-boot:run -Dspring-boot.run.profiles=mysqldev
docker exec -i iw3-gas-mysql mysql -uroot -proot iw3_db < docs/sql/seed-dev-users.sql
TOKEN=$(curl -s -X POST localhost:8080/api/v1/login -d username=operador -d password=operador123)
curl -i -H "Authorization: Bearer $TOKEN" localhost:8080/api/v1/orders?size=1        # 200
curl -i -X POST -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" -d '{"preset":1}' \
  localhost:8080/api/v1/integration/sap/orders                                       # 403
```

Usuarios de prueba (**solo desarrollo**, contraseña `<usuario>123`): `admin`, `supervisor`, `operador`, `sap`, `tms`, `loader`.

7. Limitaciones y pasos manuales

- **Base de la VM:** hoy solo tiene `martin` (`ROLE_ADMIN`), que sigue funcionando en todo. Crear los otros usuarios y roles ahí es un **paso manual con autorización** (con `htpasswd -nbBC 10`, como se hizo para `martin`; no usar el seed, que es de desarrollo).
- **401 sin JSON estándar:** el rechazo por falta de token sigue con el formato de Spring. Se revisa en la Fase 3 junto con CORS (H-30).
- **Un rol, una persona:** no hay administración de usuarios por API; los usuarios se crean por SQL.
- El `PDF` de la fase no se generó todavía (regla 8 del `CLAUDE.md`).

8. Nota sobre el repositorio

Al empezar la fase, `git fetch` mostró que `origin/main` había cambiado: el PR #16 («Fase 5 conciliación», de Benjamín Alasino, 05/11) se había mergeado desde una rama con la historia anterior a la limpieza. `origin/main` quedó con 567 commits (120 con `Co-Authored-By: Claude`) y **sin** el contenido del cierre del parcial (`/demo` cerrado, Swagger completo, Postman 07, chequeo del deploy), y el workflow de deploy corrió en verde sobre ese commit.

Esta rama partió del `main` local (317 commits, sin trailers), que es la versión entregada. Como su historia es distinta de la de `origin/main`, un PR directo daba **conflictos en 7 a 10 archivos**. Se resolvió sin reescribir nada: `restaurar-cierre-parcial` (un commit que restituye los 12 archivos) y `merge/final-fase-1-roles` / `merge/final-fase-2-alarmas`, que reaplican los commits sobre `origin/main` con **el mismo árbol** (verificado) y sin conflictos. Todo entró en `main` con el PR #18 (12/11/2026). Las ramas originales se borraron. Queda abierta la cuestión de los 120 commits con trailers (ver `CLAUDE.md`, sección 2).

9. Guía de defensa

1. **¿Qué diferencia hay entre autenticación y autorización?** Autenticación: quién sos (login, JWT). Autorización: qué podés hacer (roles). 401 = no estás autenticado; 403 = lo estás, pero tu rol no alcanza.
2. **¿Dónde se leen los roles?** Están dentro del JWT (claim `roles`, lo escribe el login en `AuthRestController`); `JWTAuthorizationFilter` los convierte en `SimpleGrantedAuthority`.
3. **¿Cómo funciona `@PreAuthorize`?** Spring envuelve el controller en un proxy; antes de ejecutar el método evalúa la expresión contra las autoridades del `SecurityContext` y, si falla, lanza `AccessDeniedException`.
4. **¿Por qué el `try/catch` del controller no atrapa ese 403?** Porque la excepción se lanza en el proxy, antes de entrar al método; la atrapa el `ExceptionTranslationFilter`.
5. **¿Para qué sirve `RestAccessDeniedHandler`?** Para que el 403 salga con el JSON estándar de la API (`message`, `code`, `devInfo`) y el frontend pueda mostrar el mensaje.
6. **¿Por qué `hasAnyRole` y no una regla por URL?** Es el enfoque de la cátedra (clase 9) y deja el permiso al lado del endpoint.
7. **¿Qué roles hay y por qué algunos son «de sistema»?** SAP, TMS y LOADER son los sistemas externos del proceso; cada uno solo puede llamar a sus endpoints. ADMIN, SUPERVISOR y OPERADOR son personas.
8. **¿Quién puede cerrar una orden y por qué?** ADMIN, SUPERVISOR, OPERADOR y LOADER: el cierre lo hace el sistema de carga, pero también una persona desde la interfaz.
9. **¿Qué pasa con un token válido de un rol que no está en la matriz?** 403 en todos los endpoints (probado con `ROLE_USER` y con un rol inventado).
10. **¿Por qué un cuerpo mal formado da 400 y no 403?** Porque el enlace del JSON ocurre antes de invocar el método; es un detalle conocido y no filtra información.
11. **¿Cómo se probó?** 98 tests con JWT reales firmados con la clave del proyecto, una matriz de 72 casos contra el backend en marcha y 32 pedidos Postman nuevos.
12. **¿Por qué el mensaje del 403 no dice qué rol falta?** Para no dar pistas a quien prueba permisos.
13. **¿Qué cambia para el frontend?** Debe leer `roles` del token, ocultar lo que el rol no permite y manejar el 403 sin cerrar la sesión (Fase 5 del frontend).
14. **¿Qué quedó pendiente?** Crear usuarios en la VM, el formato JSON del 401 y CORS (Fase 3).

10. Lista de comprobación para rendir

- [ ] Explico la diferencia 401/403 con un ejemplo.
- [ ] Recorro el pedido de la sección 3 paso por paso, con archivo y línea.
- [ ] Muestro la matriz y justifico cada fila.
- [ ] Explico por qué el 403 necesita un handler propio.
- [ ] Sé decir cómo se probó y qué números dieron.